

Episode 02: Unbekannte systematische Fehler

Oder: Warum alle Modelle falsch sind, aber manche doch
nützlich sind

Manfred Wiessner

22. September 2026

1 Einleitung

Sprecherskript & Visualisierung

[00:00–01:30] Begrüßung & Problemstellung

Kamera: Moderator direkt in die Kamera, ruhiges Bild, evtl. Einblendung des Episodentitels als Text-Overlay.

Sprecher: Willkommen zu Folge 2! Heute wollen wir uns einem Thema widmen, das in der Praxis nur allzu gerne heruntergespielt oder totgeschwiegen wird: Wie gehen wir sinnvoll mit *unbekannten* systematischen Fehlern um?

Das Thema ist enorm umfangreich, und ich kann in dieser Folge leider nicht alle Methoden behandeln, die es hierzu gibt. Ich bitte euch da um ein wenig Geduld – wenn zu einem bestimmten Teilbereich Interesse besteht, mache ich dazu aber sehr gerne noch ein eigenes Video!

Bevor wir aber tief einsteigen, müssen wir uns erst einmal eine grundlegende Frage stellen: Wie entstehen systematische Fehler überhaupt?

2 Wie entstehen unbekannte systematische Fehler?

Sprecherskript & Visualisierung

[01:30–04:30] Modelllücken, Konditionierung & ELS-Framework

Kamera: Moderation. Bei Nennung des Zitats von George Box Einblendung der Textgrafik. Später schematische Darstellung: Modell vs. Realität (Residuen).

Sprecher: Der Statistiker George Box hat es einmal perfekt auf den Punkt gebracht: „*Alle Modelle sind falsch, aber manche sind nützlich.*“

Warum ist das so? Jedes physikalische oder mathematische Modell hat seine Lücken. Es beschreibt immer nur einen vereinfachten Teil der komplexen Wirklichkeit. Genau diese Abweichungen zwischen unserem Vereinfachungsmodell und den echten Daten sind die *unbekannten systematischen Fehler*.

Jetzt könnte man natürlich versuchen, das Modell einfach beliebig kompliziert zu machen, um die Wirklichkeit genauer abzubilden. Aber das hilft leider überhaupt nichts! Denn komplizierte Modelle brauchen immer mehr Modellparameter. Bei der Analyse von Messdaten führt das sehr schnell zu ****schlecht konditionierten Gleichungssystemen****.

Die Folge: Die Aussagekraft der Parameter schwindet drastisch, weil viele völlig unterschiedliche Kombinationen von Parametern das Problem auf einmal gleich gut beschreiben.

Genau aus diesem Anlass heraus haben wir ein ****gesamtes Framework**** zu dieser Themenproblematik entwickelt und veröffentlicht. Da unser gesamter Code vollständig als Open Source bereitsteht, könnt ihr unser Konzept sehr gerne für eure eigenen Analysen und Projekte verwenden. Über ein Zitat unserer Arbeit würden wir uns dabei natürlich riesig freuen!

3 Wie kann man so was beschreiben? (Der bayesianische Ansatz)

Sprecherskript & Visualisierung

[04:30–08:00] Bayes-Modellierung, Nuisance Parameter & Begriffe

Kamera: Moderator in Großaufnahme. Bei Erwähnung der Fachbegriffe (Bayes, MCMC, Marginalisierung) dezente Einblendung von Begriffskarten am Bildrand. Später Schemadiagramm: Aufweitung der Posterior-Verteilung.

Sprecher: Wenn wir diesen unbekannten systematischen Fehler mathematisch fassen wollen, gibt es einen extrem eleganten Weg aus der bayesianischen Statistik: Wir führen sogenannte **Nuisance Parameter** (also Störparameter) ein. Ihr werdet aber gleich sehen, dass unser eigener, neuer ELS-Ansatz aus dem Paper noch viel mächtiger ist – denn wir modellieren die Residuen beziehungsweise die Standardabweichungen der Residuen direkt explizit mit.

Nun aber zuerst zum klassischen bayesianischen Ansatz: Anstatt so zu tun, als wüssten wir die exakte Form des Bias nicht, fügen wir unserem Modell einen zusätzlichen Parameter δ hinzu, der genau diesen unbekannten Drift oder Offset repräsentiert. Weil wir vor der Messung aber nicht genau wissen, wie groß δ ist, weisen wir ihm einen sogenannten **Prior** zu – zum Beispiel eine Normalverteilung um 0 mit einer realistischen Standardabweichung.

Jetzt fällt mir natürlich auf: Das sind extrem viele schwere Fachbegriffe auf einmal! Wenn wir ehrlich sind, stecken hier gleich drei eigene Riesenthemen drin, die ich in diesem Video leider unmöglich alle im Detail behandeln kann:

- **Bayes-Statistik:** Eine grundlegend alternative Sichtweise zur klassischen (frequentistischen) Statistik. Hier behandeln wir Parameter nicht als feste, unbekannte Zahlen, sondern als Wahrscheinlichkeitsverteilungen.
- **Markov Chain Monte Carlo (MCMC):** Ein mächtiges numerisches Verfahren, um stochastische Ketten zu simulieren und so ein exaktes Abbild hochdimensionaler Verteilungen zu bekommen.

- **Marginalisierung:** Das mathematische Verfahren, bei dem wir über alle möglichen Zustände unseres Störparameters δ drüberintegrieren (oder aufsummieren), um ihn aus dem Endergebnis herauszurechnen.

Ich bitte euch da um Verständnis, dass wir hier nicht in jedes Kaninchenloch abbiegen können. Falls ihr aber ein eigenes Grundlagenvideo zu Bayes-Statistik oder MCMC-Sampling wollt, schreibt es mir unbedingt in die Kommentare – dann mache ich dazu sehr gerne ein separates Video!

Was ist jetzt aber der entscheidende Vorteil dieses bayesianischen Tricks?

Durch die Marginalisierung des Nuisance-Parameters δ verschwindet der Bias zwar aus unserem Fokus, aber seine Unsicherheit bleibt erhalten! Das mathematische Ergebnis ist faszinierend: **Unsere Erwartungsintervalle (die Posterior-Intervalle) weiten sich automatisch auf.**

Wir lügen uns also nicht mehr in die eigene Tasche mit zu engen Fehlerbalken, sondern das Modell spiegelt die ehrliche, erweiterte Unsicherheit wider, die durch den unbekannten systematischen Fehler entsteht.

4 Ja, und wenn ich eine verlässliche Fehlerabschätzung mitmachen möchte? – Extended-Least-Squares

Sprecherskript & Visualisierung

[08:00–12:00] Unser Framework: Simultane Parameter- & Fehlermodellierung

Kamera: Moderator. Sobald die Formeln erwähnt werden, Schaltung auf Bildschirmübertragung/Einblendung der mathematischen Herleitung aus eurer Publikation.

Sprecher: Jetzt kommen wir zum Herzstück und zu der Lösung, die wir selbst entwickelt und publiziert haben: Was machen wir, wenn wir nicht nur die Modellparameter bestimmen wollen, sondern gleichzeitig eine ehrliche, mathematisch fundierte **Residuenabschätzung** für unsere Messdaten benötigen?

Der Schlüssel dazu ist: Wir modellieren die Unsicherheit – also die Standardabweichungen – der Residuen einfach mit! Wir führen neben den reinen Modellparametern $\vec{\xi}_1$ einen zweiten Satz an Parametern $\vec{\xi}_2$ ein, der das Verhalten der Standardabweichungen der Residuen selbst beschreibt – unser sogenanntes **Fehlermodell** $s_{c,i}(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2)$.

Ausgehend von einer Gaußschen Likelihood-Funktion in der Bayes-Statistik transformieren wir das Problem der Maximierung der Log-Posterior-Verteilung in eine Minimierung einer erweiterten Zielfunktion Z . Indem wir die Log-Posterior mit (-2) multiplizieren, bringen wir das Problem in eine Form, die der klassischen Least-Squares-Summe stark ähnelt:

$$\begin{aligned} Z &= -2 \left[\ln P \left(\{\vec{\xi}_1, \vec{\xi}_2\} \mid \{(x_i, y_i)\}, M \right) \right. \\ &\quad \left. + \ln P \left(\{(x_i, y_i)\} \mid M \right) + \frac{n}{2} \ln(2\pi) \right] \\ &= \sum_{i=1}^n \left\{ 2 \ln \left[s_{c,i} \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right) \right] + \frac{\left[y_i - y_{c,i}(\vec{\xi}_1) \right]^2}{s_{c,i}^2 \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right)} \right\} \\ &\quad - 2 \ln P \left(\{\vec{\xi}_1, \vec{\xi}_2\} \mid M \right) \end{aligned}$$

Wenn wir nun einen flachen, nicht-informativen Prior annehmen ($P(\vec{\xi} | M) = 1$), wird der hintere Logarithmus zu Null. Es bleibt unsere vereinfachte Zielfunktion Z_1 übrig:

$$Z_1 = \sum_{i=1}^n \left\{ 2 \ln \left[s_{c,i} \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right) \right] + \frac{\left[y_i - y_{c,i}(\vec{\xi}_1) \right]^2}{s_{c,i}^2 \left(y_{c,i}(\vec{\xi}_1), \vec{\xi}_2 \right)} \right\}$$

Schauen wir uns diese Gleichung Z_1 einmal ganz genau an!

Der zweite Term in der Summe ist die gewöhnliche, gewichtete Abweichungssumme $\frac{(y_i - y_{c,i})^2}{s^2}$. Wenn wir nur diesen Term minimieren würden, würde der Algorithmus versuchen, s einfach unendlich groß zu machen, um die Abweichung auf Null zu drücken – das System würde schlichtweg kollabieren.

Genau das verhindert der erste Term: $2 \ln(s_{c,i})$. Dieser logarithmische Term ist die mathematische Strafe (die *Penalty*) für zu große Unsicherheiten!

Durch die Minimierung von Z_1 führen wir ein **Extended Least Squares (ELS)** aus, das äquivalent zur Maximum-Likelihood-Schätzung und zur MAP-Lösung (*Maximum A Posteriori*) ist.

Der riesige Vorteil für die Praxis: Wir schätzen nicht nur die physikalischen Parameter $\vec{\xi}_1$ extrem robust ab, sondern erhalten über die **Hesse-Matrix** der Zielfunktion direkt die vollständige Kovarianzmatrix – also alle Fehlergrenzen, Korrelationen der Parameter und glaubwürdige Intervalle (*Credible Intervals*) für unsere Vorhersagen.

Wie das in der Praxis – zum Beispiel bei der Bestimmung von Versetzungsdichten in der Materialwissenschaft oder bei der Analyse von Spektren – in Python implementiert wird, schauen wir uns jetzt im Code an!

5 Praxisbeispiel: Gefügeveränderung beim Anlassen von Edelstahl

Sprecherskript & Visualisierung

[12:00–16:30] Anwendungsbeispiel: Versetzungsdichte in Chromstahl (N404)

Kamera: Moderator. Bei Erwähnung der Messungen Einblendung von Text-Overlays zu Legierung und Boltzmann-Funktion. Später Schaltung auf Diagramm und Tabellen.

Sprecher: Schauen wir uns das Ganze nun an einem konkreten, realen Experiment aus der Materialwissenschaft an! Wir untersuchen das Temperaturverhalten der Versetzungsdichte ρ im Martensit während des Anlassens eines Chrom-Edelstahls – der Stahlsorte Böhler N404. Die Versetzungsdichte bestimmt maßgeblich die mechanischen Eigenschaften wie Zugfestigkeit und Zähigkeit. Aus In-situ-Röntgendiffraktometrie-Daten wurde der Abbau dieser Versetzungen über die Temperatur ermittelt. Physikalisch lässt sich dieser Vorgang durch thermisch aktivierte Ausheilprozesse beschreiben, die eine typische S-förmige Kurve zeigen.

Als phänomenologisches Modell nutzen wir eine **Boltzmann-Funktion**:

$$\rho(\vartheta) = \rho_2 + \frac{\rho_1 - \rho_2}{1 + \exp\left(\frac{\vartheta - \vartheta_0}{\Delta\vartheta}\right)}$$

Hierbei sind ρ_1 und ρ_2 die Anfangs- und Endversetzungsdichte, ϑ_0 die Schwerpunkttemperatur und $\Delta\vartheta$ das Temperaturintervall des Übergangs.

Jetzt kommt die entscheidende Frage für unser ELS-Framework: **Wie wählen wir die Fehlerfunktion s_c ?**

Überstreicht ein Messwert wie die Versetzungsdichte mehrere Größenordnungen (von 10^{13} bis 10^{15} m^{-2}), führt eine konstante Unsicherheit $s_c = s_0$ oder eine Wurzel-Funktion $s_1\sqrt{\rho}$ bei hohen Temperaturen oft zu einem mathematischen Desaster: Das Konfidenz- bzw. Credible-Intervall schlägt ins Negative aus! Negative Versetzungsdichten sind physikalisch aber unmöglich.

Die Lösung: Wir wählen ein **lineares Fehlermodell**, das mit der Dichte skaliert:

$$s_c(\vartheta) = s_0 + s_1 \cdot \rho(\vartheta)$$

Mit unserem ELS-Ansatz schätzen wir nun ****alle 6 Parameter simultan**** ab – die 4 physikalischen Parameter der Boltzmann-Funktion sowie die 2 Parameter s_0 und s_1 unseres Fehlermodells!

Schauen wir auf die Ergebnisse:

- Wir erhalten extrem präzise Schätzwerte wie $\vartheta_0 = 543,2 \pm 1,4^\circ\text{C}$ und $\Delta\vartheta = 19,2 \pm 1,3\text{ K}$.
- Über die ****Hesse-Matrix**** der Zielfunktion Z_1 berechnen wir direkt die vollständige Korrelationsmatrix $\vec{C}$.
- Wir erkennen z. B. eine deutliche Antikorrelation von $-0,75$ zwischen $\Delta\vartheta$ und ρ_2 sowie von $-0,92$ zwischen den Fehlerparametern s_0 und s_1 .

Das Ergebnis ist ein physikalisch konsistentes, ehrliches Unsicherheitsband ($\pm 1s_c$), das bei allen Temperaturen strikt positiv bleibt und die experimentellen Streuungen perfekt einfängt!

6 Vollstaendiges Python-Skript

Das folgende Skript führt den Extended-Least-Squares-Fit für den Festigkeitsabbau von Edelstahl (N404) durch, berechnet die Hesse-Matrix sowie Parameterkorrelationen und visualisiert die Ergebnisse inklusive des Unsicherheitsbandes und der Residuen.

Listing 1: Vollstaendiger Quellcode zur ELS-Analyse und Unsicherheitsberechnung

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
import numdifftools as nd
```

```
#
```

```
# 1. MESSDATEN UND INITIALE PARAMETER
```

```

#
x_data = np.array([
    22, 22, 27.5, 42.5, 50, 50, 62.5, 87.5, 100, 100,
    112.5, 137.5,
    150, 150, 162.5, 187.5, 200, 200, 212.5, 237.5,
    250, 250, 262.5,
    287.5, 300, 300, 312.5, 337.5, 350, 350, 362.5,
    387.5, 400, 400,
    412.5, 437.5, 450, 450, 462.5, 487.5, 500, 500,
    512.5, 537.5, 550,
    550, 562.5, 587.5, 600, 600, 612.5, 637.5, 650
])

# Skalierung der y-Daten um Faktor 100 fuer numerische
  Stabilitaet
scaling_factor = 0.01
y_data_original = np.array([
    780.736, 694.997, 634.522, 710.078, 652.855,
    732.355, 732.608,
    660.579, 640.615, 706.121, 721.219, 729.574,
    701.986, 721.649,
    629.576, 641.398, 660.333, 660.576, 647.118,
    672.643, 716.937,
    646.079, 635.233, 618.923, 620.085, 621.435,
    609.958, 630.906,
    634.038, 621.224, 647.268, 621.815, 644.310,
    699.290, 628.957,
    642.193, 624.068, 610.783, 607.593, 605.209,
    578.407, 517.969,
    570.978, 513.906, 291.448, 216.813, 175.107,
    113.919, 73.087,
    72.024, 47.622, 45.960, 43.852
])
y_data = y_data_original * scaling_factor

# Startwerte: P0 = [rho1, rho2, theta0, dtheta, s0, s1]
P0 = np.array([6.59117, 0.40532, 543.122, 19.199,
    0.04120, 0.08803])
labels = ['rho1', 'rho2', 'theta0', 'dtheta', 's0',
    's1']

```

```

# Parameter-Bounds fuer L-BFGS-B Optimizer
bounds = [
    (1, 10),          # rho1
    (0.01, 2),        # rho2
    (100, 600),       # theta0
    (1, 100),         # dtheta
    (0.001, 0.1),     # s0
    (0.001, 10)       # s1
]

#
=====
# 2. ZIELFUNKTION S DEFINITION (Negative
  Log-Likelihood)
#
=====
def objective_s(P):
    """Berechnet die Zielfunktion S fuer das
        ELS-Framework."""
    rho1, rho2, theta0, dtheta, s0, s1 = P

    # Boltzmann-Modell
    y_model = rho2 + (rho1 - rho2) / (1 +
        np.exp((x_data - theta0) / dtheta))

    # Lineares Fehlermodell s_c = s0 + s1 * y_model
    sigma = s0 + s1 * y_model

    # Strafterm fuer physikalisch unmoeegliche Parameter
    if np.any(sigma <= 1e-9) or dtheta <= 1e-9:
        return 1e15

    # Zielfunktion S
    term1 = 2 * np.sum(np.log(sigma))
    term2 = np.sum(((y_data - y_model) / sigma) ** 2)
    return term1 + term2

```

```

#
=====
# 3. OPTIMIERUNG UND ERROR ANALYSIS
#
=====
def run_els_fit():
    print("---- STARTING ELS MINIMIZATION ----")
    result = minimize(
        objective_s, P0, method='L-BFGS-B',
        bounds=bounds, options={'ftol': 1e-9}
    )
    P_fit = result.x
    S_min = result.fun

    print(f"Status: {result.message}")
    print(f"Minimum S-Wert: {S_min:.6f}\n")

    # Hesse-Matrix und Varianz-Kovarianz-Matrix
    H_num = nd.Hessian(objective_s)(P_fit)
    V_num = np.linalg.inv(H_num)
    std_err = np.sqrt(np.diag(V_num))
    R_matrix = V_num / np.outer(std_err, std_err)

    print("Optimierte Parameter und Standardfehler:")
    for l, val, err in zip(labels, P_fit, std_err):
        print(f"  {l:7s} = {val:10.5f} +/- {err:8.5f}")

    print("\nKorrelationsmatrix R:")
    print(pd.DataFrame(R_matrix, index=labels,
        columns=labels).round(4))

    return P_fit, std_err, R_matrix

#
=====
# 4. VISUALISIERUNG DER ERGEBNISSE (PLOTING)
#
=====
def plot_results(P_fit, filename: str =
    "els_fit_n404.png"):

```

```

rho1, rho2, theta0, dtheta, s0, s1 = P_fit

x_line = np.linspace(0, 670, 500)
y_model_line = rho2 + (rho1 - rho2) / (1 +
    np.exp((x_line - theta0) / dtheta))
sigma_line = s0 + s1 * y_model_line

# Modellwerte an Messpunkten
y_model_pts = rho2 + (rho1 - rho2) / (1 +
    np.exp((x_data - theta0) / dtheta))
residuals = y_data - y_model_pts

fig, (ax1, ax2) = plt.subplots(
    2, 1, figsize=(9, 7), sharex=True,
    gridspec_kw={'height_ratios': [3, 1]}, dpi=150
)

# Haupt-Plot: Messdaten & Fit
ax1.scatter(x_data, y_data, color='#1f77b4',
    alpha=0.7, label='Messdaten N404', zorder=3)
ax1.plot(x_line, y_model_line, color='black',
    lw=2, label='ELS Boltzmann-Fit')
ax1.fill_between(
    x_line, y_model_line - sigma_line,
    y_model_line + sigma_line,
    color='gray', alpha=0.3, label=r'Streuband
    $\pm 1 s_{\mathrm{c}}$')
)
ax1.set_ylabel(r'Versetzungsdichte $\rho$
    [$10^{14}\backslash,\mathrm{m}^{-2}$'])
ax1.set_title('ELS-Fit der Versetzungsdichte',
    fontsize=12, fontweight='bold')
ax1.grid(True, linestyle='--', alpha=0.5)
ax1.legend()

# Residuen-Plot
ax2.axhline(0, color='black', linestyle='--', lw=1)
ax2.scatter(x_data, residuals, color='red',
    alpha=0.7)
ax2.set_xlabel(r'Temperatur $\vartheta$
    [$^\circ\mathrm{C}$'])

```

```

ax2.set_ylabel('Residuen')
ax2.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig(filename)
plt.close()
print(f"\n[INFO] Diagramm gespeichert unter:
      {filename}")

#
=====
# EXECUTION
#
=====
if __name__ == "__main__":
    P_fit, std_err, R_matrix = run_els_fit()
    plot_results(P_fit)

```

7 Fazit & Abspann

Sprecherskript & Visualisierung

[16:30–18:00] **Fazit: Die Stärken der Extended Least Squares (ELS) Methode**

Kamera: Moderator in Großaufnahme, kurze Texteinblendungen zu den Kernvorteilen der ELS-Methode.

Sprecher: Fassen wir die wichtigsten Erkenntnisse unserer heutigen Episode noch einmal zusammen!

Die ****Extended Least Squares (ELS)**** Methode ist ein extrem mächtiges Werkzeug in der wissenschaftlichen Datenanalyse und Ingenieurpraxis. Ihr entscheidender Vorteil: Sie ermöglicht es uns, ****Modellparameter und die Standardabweichungen der Modellprognosen vollkommen simultan zu berechnen****.

Warum solltet ihr ELS in euren eigenen Projekten einsetzen?

1. **Breite Anwendbarkeit:** Die ELS-Methode lässt sich universell auf eine Vielzahl von Optimierungs- und Regressionsproblemen anwenden – weit über die Materialwissenschaft hinaus.
2. **Ideal bei kleiner Datenbasis:** Sie ist besonders wertvoll, wenn wir mit einer begrenzten Anzahl an Messdaten arbeiten müssen, da sie sämtliche verfügbaren Informationen im Datensatz optimal verwertet.
3. **Robuste Gewichtung:** Sie liefert auch dann stabile und verlässliche Schätzungen, wenn starke Schwankungen in den Gewichtungsfaktoren oder Unsicherheiten der Messdaten auftreten.

Anstatt Unsicherheiten als störendes Übel zu betrachten, macht das ELS-Framework sie zu einem integralen Bestandteil der Modellierung. Ihr erhaltet mathematisch fundierte, ehrliche Konfidenzintervalle und schützt euch zuverlässig vor unphysikalischen Ergebnissen!

Vielen Dank fürs Zuschauen! Wenn euch diese Folge weitergeholfen hat, lasst gerne ein Like da und abonniert den Kanal. Den vollständigen Python-Code und die Datensätze findet ihr wie gewohnt unten in der Videobeschreibung zum Nachrechnen. Bis zum nächsten Mal!